

Kim SeongGyu (김성규)

재고 · 예약 · 외부 채널 데이터 · LLM을 이미 프로덕션에서 다뤄본 백엔드 엔지니어

숙박 ARI(재고·요금·가용성) · 채널 매니저 연동 · 예약 동시성 · LLM 활용

= 제가 Upbrella(3년 운영) · Cove(외주) 에서 만난 문제와 구조가 같은 문제입니다.

Email gseonggyu968@gmail.com 지원 트립비토즈 · 호텔(B2B) 시스템 백엔드 개발

프로필 — 재고·외부 데이터·LLM을 운영해본 백엔드 엔지니어

"학교 과제가 아니라 사용자 700명+이 쓰는 서비스를 3년째 운영하고, 외주로 다중 거래소 데이터 파이프라인과 LLM 기능을 프로덕션에 올려봤습니다."

핵심 강점 (트립비토즈 주요업무 기준)

- 재고 동시성을 코드로 해결 — 우산 1개가 동시에 두 명에게 나가는 race condition을 비관적 락 + 경로 분리로 차단 (객실 overbooking과 같은 구조)
- 외부 채널 데이터 파이프라인 운영 — Cove에서 Binance·Upbit 다중 거래소 시세를 수집·정규화·적재 (채널 매니저 연동과 같은 성격)
- LLM을 제품 기능으로 — OpenAI GPT로 뉴스 요약·감성 분석·경제 캘린더 번역을 실제 서비스 기능으로 운영
- Aurora/MySQL 대용량 모델링 — 5,000만 건 시계열을 복합 인덱스·파티셔닝·커서 페이지네이션으로 튜닝
- AWS ECS·Lambda 직접 운용 — Python Lambda 7종(Docker/ECR), 멀티모듈로 배치를 API 서버와 격리

이력

- 업브렐라 (Upbrella) · Backend
2023.07 ~ 현재 (3년) · 운영 중 · 500 커밋
- 코브 (Cove) · Backend 외주
2024.02 ~ 2024.10 · Java21·SB3.2·Redis
- 우아한테크캠프 7기 TA
2024.06~08 · 코드 리뷰 및 QA 담당
- 코드스쿼드 백엔드 마스터즈
2023.01~06 · 리뷰·페어 프로그래밍

트립비토즈가 푸는 문제 — **재고 · 예약 · 외부 채널 · LLM** — 은 제가 두 프로덕션에서 이미 만난 문제입니다. OTA 도메인만 새로 배우면 됩니다.

목차

"내가 만든 것" 순서가 아니라, "트립비토즈의 주요업무를 나는 어떻게 풀어봤는지" 순서로 배치했습니다

01	왜 트립비토즈인가 공고의 4대 주요업무 · 자격요건 ↔ 내 경험 매핑	p.04
02	예약·재고 동시성 대규모 트래픽에서 재고 1개에 2 예약(overbooking) 방지 — 비관적 락	p.06
03	외부 채널 연동 · 데이터 처리 다중 소스 수집·정규화 파이프라인 · Python Lambda 7종	p.08
04	LLM 생태계 활용 OpenAI GPT를 제품 기능으로 운영 (Bedrock/OpenAI)	p.10
05	데이터 모델링 · 조회 성능 Aurora/MySQL 복합 인덱스·파티셔닝·커서 · N+1 · Redis	p.11
06	운영 · 인프라 · 백오피스 ECS/Lambda · CI/CD · 내가 만든 문제 · Full-Stack 갭	p.13

호텔 B2B 시스템의 주요업무를, 저는 같은 구조로 풀어봤습니다

공고 주요업무 — "예약·객실·요금·재고 API 개발 및 대규모 트래픽 환경의 동시성 제어와 예약 처리", "외부 채널(Channel Manager) API 연동 및 데이터 처리"

트립비토즈 주요업무 / 요건	제가 직접 구현·운영한 대응물	공통된 본질
예약·재고 API + 대규모 트래픽 동시성	우산 재고 동시 대여 race condition → 비관적 락 + 경로 분리	한정된 재고에 동시 예약(overbooking) 방지
숙박 ARI(가용성·요금·재고) 관리	우산 가용상태·매장별 재고·보증금(요금) 상태 관리 (상태 기계)	가용성·요금·재고의 시점별 상태 전이
외부 채널(Channel Manager) 연동·데이터 처리	Cove 다중 거래소(Binance·Upbit) 시세 수집·정규화·적재	외부 피드를 신뢰 가능한 단일 상태로 통합
LLM 생태계(Bedrock, OpenAI) 활용	Cove OpenAI GPT — 뉴스 요약·감성분석·경제캘린더 번역 (제품 기능)	LLM을 제품 기능으로 통합·운영
통합 백오피스(Full-Stack)	Upbrella 어드민 대여 이력·통계 API (N+1 → 1 쿼리)	운영 의사결정을 받치는 조회 성능
클라우드(AWS ECS·Lambda) 운영	Python Lambda 7종(Docker/ECR) · ECS · CodeDeploy 직접	서버리스 + 컨테이너 운영

OTA 도메인은 새로 배워야 하지만, "재고·예약·외부 데이터·LLM에서 무엇이 위험한가"를 알아보는 감각 은 이미 있습니다.

자격요건 — 정직한 자기 평가

코드·커밋으로 직접 입증할 수 있는 것과, 아직 미적용인 것을 나눠 적었습니다

● 직접 수행으로 입증 가능

자격요건 / 우대	증거
Java / Spring 1~5년	Cove(Java 21·SB 3.2) · Upbrella(Kotlin·SB 2.7) 3년
RESTful API 설계·운영	REST Docs 기반 설계·문서화 · 2 프로덕션 운영
RDBMS(Aurora/MySQL) 모델링	MySQL 8.0 · 복합 인덱스·파티셔닝·5,000만 건 튜닝
클라우드(AWS ECS·Lambda)	Lambda 7종(Docker/ECR) · ECS · CodeDeploy 직접
LLM 생태계(OpenAI API)	OpenAI GPT 제품 기능 운영 (요약·감성·번역)
동시성 제어·예약 처리 로직	비관적 락 + 경로 분리 — 재고 정합성 (p.06-07)
협업·커뮤니케이션	500 커밋 PR 리뷰 · 캠프 TA 코드 리뷰
[우대] Python·Kotlin 다언어 AI	Cove Python Lambda + Kotlin — 다언어 환경 운영

● 아직 미적용 — 솔직하게

• AWS Bedrock

특정 서비스는 미사용. 다만 OpenAI API는 제품 기능으로 운영 — LLM 연동·프롬프트·비동기 처리 패턴은 동일 (p.10)

• [우대] React / Redux 프론트엔드

미보유. 통합 백오피스 Full-Stack은 오히려 도전 과제로 반가움 (p.15)

• [우대] OTA·여행·숙박 도메인

미경험. 다만 재고·예약·정산의 본질은 두 프로덕션에서 겪음

자격요건의 "AWS Bedrock, OpenAI API 등 LLM 생태계"에서 '등'이 반가웠습니다 — 특정 벤더가 아니라 LLM을 제품에 녹이는 감각이 핵심이라 생각합니다.

재고 1개가 동시에 두 예약에 나가는 문제

트립비토즈로 치면 — 마지막 1개 남은 객실에 두 예약이 동시에 들어오는 overbooking 상황

순진한 구현에는 '틈'이 있습니다

1. 재고가 예약 가능한가?
(`rentable = true` 확인)

← 여기에 틈이 있다
2. 가능하면 → 예약 처리
(`rentable = false`, History 생성)

두 요청이 동시에 1번을 통과하면

- 둘 다 "예약 가능"이라는 답을 받고 **둘 다 2번을 실행**
- 물리적으로 존재하지 않는 재고가 팔림 — 객실은 하나인데 두 예약이 확정됨 (overbooking)
- 성수기·인기 숙소처럼 요청이 몰리는 순간에 정확히 터짐
- 데이터가 적을 땐 **절대 드러나지 않음** — 로컬에서는 재현조차 안 됨

왜 이 문제가 이 도메인에서 특히 치명적인가

- 물리 재고(객실/우산)가 걸려 있어 롤백이 불가능합니다. DB는 되더라도 이미 확정된 예약을 되돌리는 건 곧 고객 사고입니다.
- 금전(요금/보증금)이 얹혀 있어 "누가 진짜 예약자인가"가 곧 환불·책임 소재 분쟁으로 직결됩니다.

비관적 락, 단 '꼭 필요한 트랜잭션에만'

```
// 읽기 경로 — 락 없음 (재고 목록 조회 등 대다수 트래픽)
fun findByIdAndDeletedIsFalse(id: Long): Optional<Umbrella>

// 예약(대여) 경로 — 비관적 쓰기 락 + 3초 타임아웃
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints(QueryHint(name = "javax.persistence.lock.timeout", value = "3000"))
fun findByIdAndDeletedIsFalseForUpdate(@Param("id") id: Long): Optional<Umbrella>
```

src/main/kotlin/upbrella/be/umbrella/repository/UmbrellaRepository.kt:15-21

Q. 왜 낙관적 락이 아닌가?

- 낙관적 락은 충돌이 드물 때 유리 — 충돌 시 예외 후 재시도
- 그런데 이 문제는 정반대. 충돌은 **성수기·인기 숙소에 집중**됩니다
- 즉 드문 게 아니라 **몰릴 때 몰립니다**
- 여기서 낙관적 락 → "예약 실패, 재시도하세요" 반복. **재고 예약에서 재시도 UX는 곧 이탈**
- 비관적 락은 잠깐 기다렸다 최신 상태를 보게 해 **"이미 예약됨"**이라는 정확한 답을 줍니다

Q. 왜 예약 경로에만?

- 락은 공짜가 아닙니다
- 조회 트래픽(재고·요금 목록)이 예약보다 **훨씬 많음**
- 여기까지 락을 걸면 **정합성이 필요 없는 곳에서 처리량만 깎임**
- 그래서 메서드 자체를 분리 — 락 없는 조회 / ...ForUpdate
- "락은 꼭 필요한 트랜잭션에만"을 **코드 구조로 강제**

Q. 왜 타임아웃 3초인가?

- 없으면 락 대기가 무한정 → **커넥션 풀 고갈**
- 결국 예약과 **무관한 API까지 함께 죽음** (장애 전파)
- 3초 = 정상 트랜잭션이면 충분 + 사용자가 기다릴 수 있는 상한
- 초과 시 **한 요청만 실패하고 시스템은 살아 있게** 하는 것이 목적

더 큰 트래픽에선 Redis 분산 락(Redisson)으로 확장 — Cove에서 Redis를 운영한 경험이 있습니다. **트레이드오프를 말할 수 없으면 선택이 아니라 답입니다.**

외부 피드를 신뢰 가능한 내부 상태로 바꾸는 파이프라인

채널 매니저 연동과 구조가 같은 문제 — Cove에서 Binance·Upbit 다중 거래소 시세를 다뤘습니다

무엇을 했나

- 다중 거래소 시세를 수집·정규화하는 비동기 처리 흐름 설계·운영
- 정기적으로 대량 데이터를 수집·적재하는 구조 (매시간 실행)
- Flyway V1~V88+ 로 스키마를 버전 관리하며 파이프라인과 함께 진화

채널 매니저와 왜 같은 구조인가

- 여러 외부 소스(거래소 = 채널)마다 스키마·레이트리밋·장애 특성이 제각각
- 이를 정규화해 단일 저장소로 통합 (coin_candles ≈ ARI 테이블)
- 정기·대량 수집, 실패 지점을 격리해 다른 소스로 전파 방지

핵심 설계 — 멀티모듈로 배치를 API 서버와 격리

cove-api-server	REST · WebSocket
cove-batch	캔들 수집 · 포인트 정산 · 가격 알람 (@Scheduled)
db-core	Entity · Repository · Flyway 공유

배치 장애가 사용자 요청 경로로 전파되지 않도록 프로세스를 격리하고, 스키마 정의를 한 곳(db-core)에 모아 모듈 간 불일치를 차단했습니다.

채널 매니저는 결국 "믿을 수 없는 외부 피드를, 믿을 수 있는 내부 상태로 바꾸는 일" — 그 파이프라인을 운영해봤습니다.

외부 데이터 처리를 요청 경로에서 분리해 서버리스로

자격요건 "클라우드(AWS, ECS, Lambda 등) 서비스 운영" · 우대 "Python 다언어 AI 서비스 구축"에 대한 답

Lambda 7종이 하는 일

- 코인 스코어링 (RSI / MACD 지표 계산)
- 모멘텀 (가중 평균 수익률)
- 뉴스 크롤링 + GPT 요약 · 감성 분석
- 경제 캘린더 GPT 번역
- → 매시간 실행되는 외부 데이터 수집·가공 작업들

운영에서 부딪힌 제약과 선택

- pandas · selenium이 **Lambda Layer 250MB 제한을 초과**
- → **Docker 이미지(ECR)** 배포로 전환해 용량 제약을 우회
- 각 작업을 독립 함수로 분리 → **Spring 서버 재배포 없이 6개까지 독립 추가**
- 외부 I/O·크롤링·LLM 호출을 **요청 경로 밖으로 빼서** API 지연·장애와 분리

7종

Python Lambda
외부 데이터 처리 함수

0회

함수 추가 시
Spring 서버 재배포

ECR

Layer 250MB 제약을
Docker 이미지 배포로 우회

외부 데이터 처리를 **독립 배포 가능한 서버리스 단위로 쪼개는 감각** — 채널 연동·데이터 처리에 그대로 쓸 구조입니다.

LLM을 '데모'가 아니라 '제품 기능'으로 운영했습니다

자격요건 — "AWS Bedrock, OpenAI API 등 LLM 생태계 활용 경험" · 우대 — "Python, Kotlin 등 다언어 환경에서의 AI 서비스 구축 경험"

① 제품 기능으로

- 뉴스 크롤링 후 GPT 요약 · 감성 분석
- 경제 캘린더 GPT 번역
- 데모가 아니라 실제 사용자에게 나가는 기능으로 운영

② 운영 관점으로

- LLM 호출을 Lambda로 분리 (비동기 성격)
- 외부 LLM의 지연·실패가 요청 경로·저장 트랜잭션을 막지 않도록 설계
- LLM은 느리고 불안정한 외부 I/O — 결합을 끊는 게 먼저라고 배웠습니다

③ 검증하며 씀 (Claude Code)

- Claude Code로 레포를 분석해 성능 리포트·기술 문서 산출
- AI가 지정한 N+1은 show_sql로 실측 확인 후에만 반영
- ZGC 도입 제안은 실측 후 "과하다"고 거절 — 거절도 결정입니다

Bedrock을 안 써봤다는 것 — 솔직하게

AWS Bedrock 자체는 미사용입니다. 다만 LLM 연동·프롬프트 설계·응답 검증·비동기 처리·실패 격리의 패턴은 동일합니다. Bedrock으로 갈아끼워도 같은 구조로 붙일 수 있습니다.

특정 벤더가 아니라, LLM을 제품에 안전하게 녹이는 감각 — 그게 이미 있다는 것이 제 차별점입니다.

대용량 재고·시계열 데이터를 조회 패턴에서 역산해 설계

ARI는 결국 대용량 재고·요금·가용성 데이터 — Cove에서 5,000만 건 시계열을 다룬 방식과 같습니다

복합 인덱스 카디널리티 설계

```
INDEX (ticker, source, unit, timestamp)
```

- 카디널리티 높은 ticker를 선두, 범위 스캔 컬럼 timestamp를 **마지막**에 배치
- 시간 파티셔닝 + 커버링 인덱스로 스캔 범위 축소 + 테이블 액세스 제거

커서 페이지네이션 + TSID

```
// LIMIT 10000, 20 → 10,020행 읽고 10,000행 폐기
WHERE id < :cursor ORDER BY id DESC LIMIT 20
```

- **O(offset)** 스캔을 스캔 범위 size로 고정
- TSID(시간 정렬 ID)로 **별도 인덱스 없이** 시간순 커서 구현

5,000만 건

coin_candles 테이블
단조 증가 시계열

배운 것

인덱스는 "겉면 빨라지는 것"이 아니라 **조회 패턴에서 역산해 설계하는 것**입니다. 단조 증가 데이터는 인덱스만으로 한계가 오고, 그때 파티셔닝이 필요해집니다. ARI 재고·요금 조회도 같은 원리입니다.

Aurora/MySQL은 "쓰면 되는" 도구가 아니라, **조회 패턴·카디널리티·파티션 프루닝까지 이해하고 설계하는 대상**입니다.

대규모 트래픽에서 DB를 지키는 두 축

쿼리 수를 줄이고(N+1), 반복 집계를 캐시로 옮긴다(Redis) — 백오피스 조회와 대규모 트래픽 모두에 필요

① N+1 — 해결책을 '하나'로 고르지 않음

- show_sql로 어드민 목록에서 **페이지당 최악 121 쿼리** 발견 (연관 6개 × 20건)
- **fetchJoin** — 엔티티 수정·도메인 로직 경로 (영속성 컨텍스트 올림)
- **@QueryProjection** — 읽기 전용 목록 (필요 컬럼만, 컨텍스트 안 올림)
- 컬렉션 N+1은 **@BatchSize(100)** — 페이징과 충돌(fetch join 불가) 회피
- → **121 → 1 쿼리**, 이후 데이터 건수와 무관한 상수

② Redis — 네 가지 목적으로 분리

- **Sorted Set 랭킹** — 3테이블 조인+GROUP BY 집계를 **요청마다** → **시간당 1회 배치**로 (최신성 1h 지연 수용)
- **Rate Limit** — @CheckAbusing AOP로 어뷰징 방지
- **세션 · 캐시** — 반복 조회 흡수
- **Pub/Sub** — WebSocket 메시지 브로커

대규모 트래픽 환경에서 배운 것 — **DB 집계 횟수를 '요청 수'에서 '시간당 1회'로 바꾸는 것**만으로도 병목이 사라집니다.
최신성 1시간 지연을 의도적으로 수용한 건, 트레이드오프를 알고 고른 선택이었습니다.

빌드부터 비용 회수까지 한 흐름으로 자동화

자격요건 "클라우드(AWS, ECS, Lambda 등)에서의 서비스 운영 경험"에 대한 답

GitHub Actions 파이프라인 (Upbrella)



AWS 직접 운용

- RDS (MySQL 8.0) — 관리형 DB
- ECS / ECR — 컨테이너 · 레지스트리
- Lambda — Python 7종 (Docker/ECR)
- CodeDeploy — 배포 자동화 · 후 롤백
- S3 · CloudFront · SSM · EventBridge

무중단성을 고려한 운용

- CodeDeploy 후에서 **kill -15** graceful shutdown 후 재기동
- **--restart unless-stopped** 자동 복구
- **image prune** 으로 디스크 누수 방지
- dev/prod 프로파일 · PR 트리거 테스트

배치를 격리한 멀티모듈 (Cove)

cove-api-server / **cove-batch** / **db-core** 로 분리해 배치 장애가 사용자 요청 경로로 전파되지 않도록 프로세스를 격리. Testcontainers·Sentry·Prometheus로 테스트·모니터링.

단순히 "배포 스크립트를 돌린다"가 아니라, **ECS·Lambda·컨테이너 생명주기·비용·장애 격리까지 이해하고 통제**하려 했습니다.

운영을 안정시키려 넣은 것이, 장애를 끌어들이고 있었습니다

```
<root level="INFO">
  <appender-ref ref="SLACK"/>  ← 모든 INFO 로그가 Slack으로
</root>
```

```
override fun append(event: ILoggingEvent) {
    sendToSlack(formatMessage(event))
    // ↑ 요청 스레드에서 동기 HTTP POST
}
```

무슨 일이 벌어지는가

- 로그 한 줄마다 Slack 왕복(수십~수백 ms)만큼 **사용자 요청이 멈춥니다**
- root가 INFO라 **Hibernate/Spring의 일상 로그까지 전부 Slack으로**
- Slack이 rate limit(429)에 걸리거나 느려지면 **그 지연이 그대로 사용자 응답 지연으로 전파**
- HttpURLConnection에 **타임아웃이 없어 무한 대기**까지 가능
- → 외부 서비스 장애가 우리 서비스 장애가 되는 **전형적인 결합**

개선 방향 (P0로 우선순위 지정)

- 레벨 분리 — Slack appender를 WARN/ERROR로 제한. **알림은 알리다워야 합니다** · 비동기화 — AsyncAppender로 감싸 요청 스레드와 분리 (neverBlock=true)
- 타임아웃 명시 — connectTimeout / readTimeout 2~3초 · 같은 분석에서 **application.properties의 운영 시크릿 평문 커밋**도 발견 → 키 전량 로테이션 권고

제가 짠 코드의 문제를 제가 찾아 P0로 올려둔 것이, 이 포트폴리오에서 **가장 정직한 부분**이라고 생각합니다.

3년 운영에서 배운 건 — **"내 코드도 의심해야 한다"**

백오피스 — 어드민은 이미, 프론트는 정직하게

주요업무 "통합 백오피스 시스템 개발 (Full-Stack 영역)" · 우대 "React, Redux 프론트엔드"

이미 한 것

- Upbrella 어드민 대여 이력·통계 API 설계·운영
- N+1 → 1 쿼리로 목록 조회 최적화 (p.12)
- 운영자용 Slack 실시간 알림 — 미환급 건수 동반 통지로 환급 누락 방지

🕒 Full-Stack — 도전 과제

- React / Redux 미보유 (우대사항)
- 공고의 "Full-Stack 영역"이 오히려 반가웠습니다 — 백엔드 중심으로 일하되 화면까지 이해하고 싶었던 방향
- 백엔드 API 설계자가 백오피스 화면을 알면 스펙 왕복이 줄어듭니다

왜 자신 있나

- 백오피스의 어려움은 화면이 아니라 "운영자가 어떤 판단을 하는가"를 아는 것
- 그건 3년 운영에서 겪었습니다 — 운영자가 무엇을 보고 싶어하고, 어떤 데이터가 없으면 사고가 나는지
- 화면 프레임워크는 배우면 되지만, 그 감각은 겪어야 생깁니다

통합 백오피스의 본질은 화면이 아니라 "운영 의사결정에 필요한 데이터를 정확하고 빠르게 보여주는 것"
— 그 조회 성능·데이터 정합성은 이미 다뤘습니다.

기술 스택

굵은 항목은 트립비토즈 공고에 명시된 요구 / 우대 기술입니다

언어 / 프레임워크

Kotlin

Java (17·21)

Spring Boot (2.7·3.2)

Spring Data JPA

QueryDSL

Python

데이터베이스

MySQL (Aurora/RDS)

Redis

복합 인덱스

파티셔닝

커버링
인덱스커서
페이지네이션

Flyway

클라우드 / 인프라

AWS ECS

Lambda

ECR

S3

RDS

CodeDeploy

CloudFront

Docker

GitHub Actions

AI / LLM

OpenAI GPT API

요약·감성분석·번역 (제품
기능)

Claude Code

모니터링 / 로깅

Datadog APM

Sentry

Micrometer/Prometheus

Actuator

nGrinder

인증 / 보안

JWT

OAuth 4종 (Strategy 패턴)

Apple ECDSA

OTP (HOTP)

테스트 / 협업

JUnit 5

Testcontainers

Fixture Monkey

REST Docs

Git

Linear

Swagger

주력은 Kotlin/Spring이고, Python·LLM·Redis·Lambda까지 — 트립비토즈가 요구하는 다언어·AI 환경을 이미 다뤄봤습니다.

트립비토즈가 저를 뽑아야 하는 이유

트립비토즈가 중요하게 볼 것	제 답
대규모 트래픽 동시성 · 예약 처리	우산 재고 동시 대여를 비관적 락 + 경로 분리로 차단 — overbooking과 같은 구조를 실제로 겪고 고침 (p.06-07)
외부 채널(Channel Manager) 연동·처리	Cove 다중 거래소 수집·정규화 파이프라인 + Python Lambda 7종 — 외부 피드를 신뢰 가능한 상태로 (p.08-09)
LLM 생태계 활용	OpenAI GPT를 요약·감성·번역 제품 기능으로 운영 — 벤더가 아니라 통합·운영 감각을 보유 (p.10)
RDBMS(Aurora/MySQL) 모델링	5,000만 건 시계열을 복합 인덱스·파티셔닝·커서로 튜닝 — 조회 패턴에서 역산해 설계 (p.11)
서비스 관점의 지속 개선	3년 운영에서 내가 만든 문제(동기 Slack 로깅)까지 찾아 P0로 문서화 — 측정 없이 "개선했다" 말하지 않음 (p.14)

정직하게 적은 것들

- **내가 만든 문제** (동기 Slack 로깅) · **아직 미적용** (AWS Bedrock) · **미보유** (React · OTA 도메인) · **보유** (동시성·채널 연동·LLM·대용량·Lambda)
- 트립비토즈가 푸는 문제의 뼈대를 이미 겪었기에, **모르는 도메인만 정확히 알고 빠르게 채우겠습니다** — 모르는 걸 모른다고 말하는 게 학습의 전제입니다.

이 포트폴리오의 모든 주장은, 레포의 **코드 · 커밋 · 실측 문서**로 그 자리에서 열어 보여드릴 수 있습니다.